

Coursework help session

Quick recap: What is a compiler?

Practically: A compiler is a program that takes source code and turns it into machine code that can run on a CPU.

Source code

```
int func(int x)
{
    z = 2
    y = x + x;
    return y;
}
```



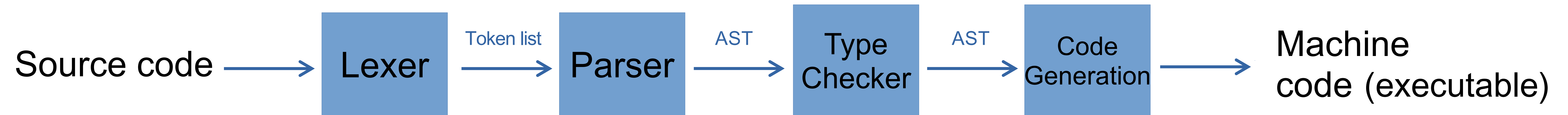
Compiler



Machine
code (executable)

```
...
li t1 2
jal func
add t2,t1,1
ret
...
```

What steps does a compiler have?

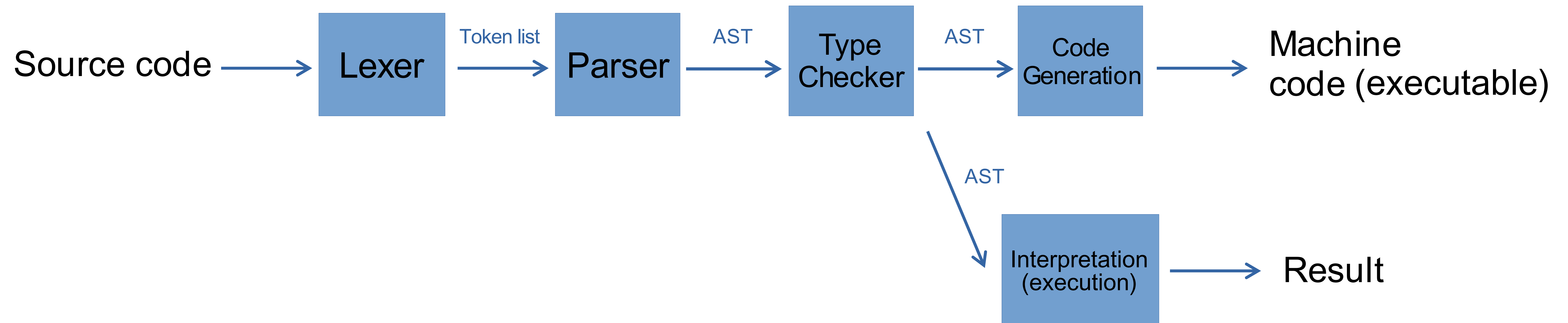


*Real world production compilers may have more steps (e.g. optimisation, intermediate representation etc)

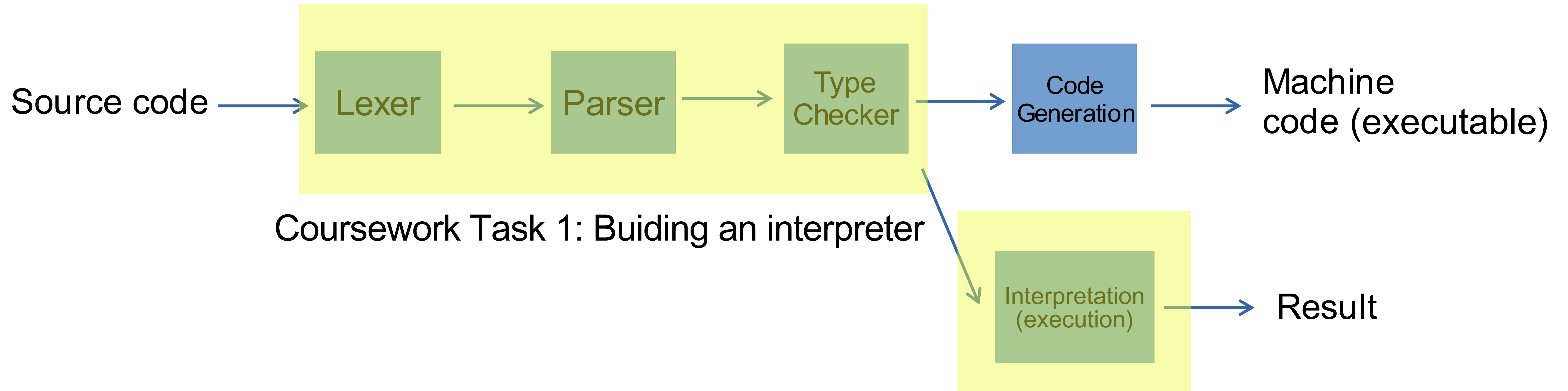
What does it take to run a program?

Two alternative approaches (see Lecture 1):

- **Compiler:** creates an executable that can be run later
- **Interpreter:** executes the program by traversing the AST



What does it take to run a program?



What is a lexer?

A lexer is a program that chops up the input into a list of specified tokens

Source code

```
{  
    a := 0;  
    i := 0  
}
```



Token list

```
LBrace  
Idfr (a)  
Assign  
IntLit (0)  
Semicolon  
Idfr (i)  
Assign  
IntLit (0)  
RBrace
```

How do specify what our tokens look like?

We can describe what each token will look like,
using Regular Expressions (regex)

Sometimes the
regex for a
token can be as
simple as just
writing the exact
string



Token class	Lexeme
While	<code>while</code>
Comma	<code>,</code>
RParen	<code>)</code>
LBrace	<code>{</code>
Semicolon	<code>;</code>
RBrace	<code>}</code>

Any variable name starting with a lower case letter



Token class	Lexeme
Idfr	<code>[a-z][A-Za-z0-9_]*</code>
IntLit	<code>0 [1-9][0-9]*</code>



zero, or any other number not starting with zero

What is a parser?

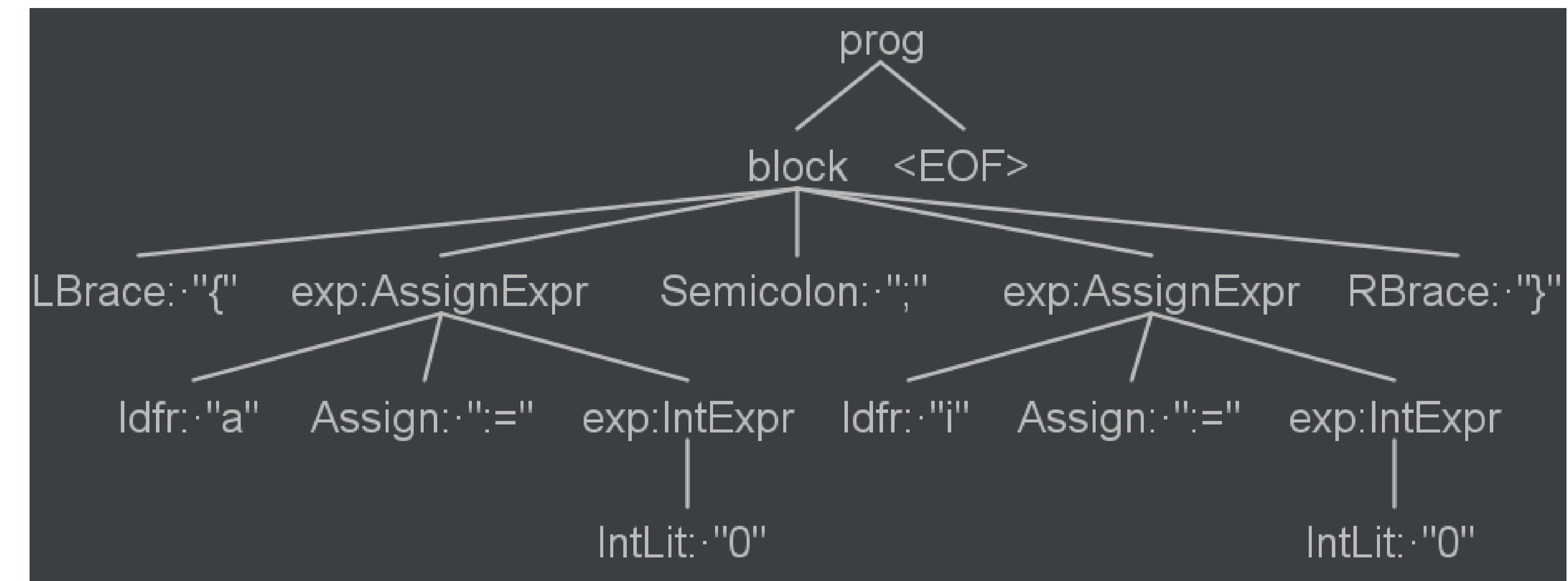
A parser makes sure that these tokens are in a valid structure, visualizes the parse tree and builds the AST to represent this structure.

Token list

LBrace
Idfr (a)
Assign
IntLit (0)
Semicolon
Idfr (i)
Assign
IntLit (0)
RBrace



Parse tree



How do we define the structure of our language?

We can describe this using Context Free Grammars (CFGs).

EXP → **IDFR := EXP**
 | **(EXP BINOP EXP)**
 | **INTLIT**

BINOP → **==** | **<** | **+**

INTLIT → (an integer)

IDFR → (a string)

In CFGs you can replace the variable on the right with any of the options (separated by ' | ') on the left

If we started with the variable EXP we could go:

EXP ⇒

IDFR := EXP ⇒

IDFR := (EXP BINOP EXP) ⇒

IDFR := (INTLIT BINOP EXP) ⇒

IDFR := (INTLIT + EXP) ⇒

IDFR := (INTLIT + INTLIT) ⇒ ...

... for example: **x := (1 + 2)**

What does ANTLR do?

ANTLR takes

- our Lexing rules (regex)
- and our Parsing rules (CFG)

and outputs a collection of Java classes that we can use to Lex/Parse input for that language.



How do we tell ANTLR our rules?

ANTLR takes the language specification in a **.g4** file containing **both** lexing rules and parsing rules

Language name

Parsing rules
(non-terminals)

Lexing rules
(terminals)

Ignore whitespace (tabs,
newlines, space), just 'skip' it

```
grammar DemoLang;

prog
  : block EOF
  ;

block
  : LBrace ene+=exp (Semicolon ene+=exp)* RBrace
  ;

exp
  : Idfr Assign exp
  | LParen exp binop exp RParen
  | While exp Do block
  | IntLit
  | Idfr
  ;

binop
  : Eq          #EqBinop
  | Less        #LessBinop
  | Plus        #PlusBinop
  ;

LParen : '(' ;
Comma  : ',' ;
RParen : ')' ;
LBrace : '{' ;
Semicolon : ';' ;
RBrace : '}' ;

Eq : '=' ;
Less : '<' ;
Plus : '+' ;

Assign : ':=' ;
While : 'while' ;
Do : 'do' ;

Idfr : [a-z][A-Za-z0-9_]* ;
IntLit : '0' | [1-9][0-9]* ;
WS : [ \n\r\t]+ -> skip ;
```

ANTLR Lexing Rules

Lexing rules follow the format of **TOKEN : REGEX ;**

Token class	Lexeme
LParen	(
Comma	,
RParen	)



```
LParen : '(' ;  
Comma  : ',' ;  
RParen : ')' ;  
LParen : '(' ;
```

Token class	Lexeme
Idfr	[a-z][A-Za-z0-9_]*
IntLit	0 [1-9][0-9]*



```
Idfr : [a-z][A-Za-z0-9_]* ;  
IntLit : '0' | [1-9][0-9]* ;  
ws : [ \t \n \r ]+ ;
```

ANTLR Parsing Rules

ANTLR **.g4** syntax can look very similar to usual CFG notation, with some differences.

```
EXP → IDFR := EXP  
    | (EXP BINOP EXP)  
    | while EXP do BLOCK  
    | IDFR  
    | INTLIT
```

```
exp  
    : Idfr Assign exp  
    | LParen exp binop exp RParen  
    | While exp Do block  
    | IntLit  
    | Idfr  
    ;
```

ANTLR Parsing Rules

Dealing with sequences

BLOCK → { ENE }

ENE → EXP | EXP; ENE



ENE → EXP | EXP; ENE
is equivalent to
ENE → EXP(; EXP)*

```
block
    : LBrace ene+=exp (Semicolon ene+=exp)* RBrace
    ;
```

```
public static class BlockContext extends ParserRuleContext {
    public ExpContext exp;
    public List<ExpContext> ene = new ArrayList<ExpContext>();
    public TerminalNode LBrace() { return getToken(DemolangParser.LBrace
```

In ANTLR you can write these two rules separated, but there is also an alternative.

Instead of separate BLOCK and ENE rules, in this case we *can* combine them. This single rule says a block contains at least one EXP, followed by 0 or more EXPs each separated by Semicolons.

The `ene +=` syntax tells ANTLR to store all these EXPs to a list called ene - we can access it in the parse tree through

`ctx.ene()`

(where ctx is a BlockContext object)

ANTLR Parsing Rules

Epsilon (the empty string, or '*nothing*') can be represented by an empty clause.
To represent having '*something-or-possibly-nothing*', you can a question mark:

```
optionalrule  
: Something  
| ; /*nothing*/
```

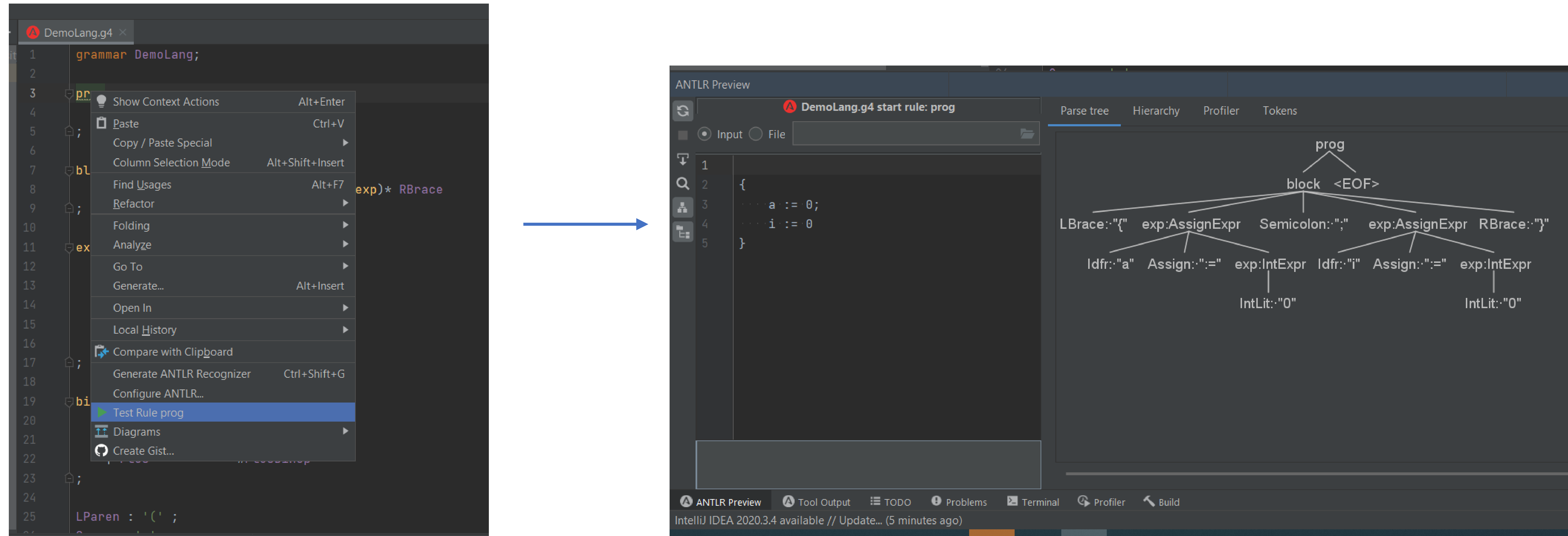
```
optionalrule  
: Something? ;
```

A full list of ANTLR Parser rule syntax can be found at:

<https://github.com/antlr/antlr4/blob/master/doc/parser-rules.md>

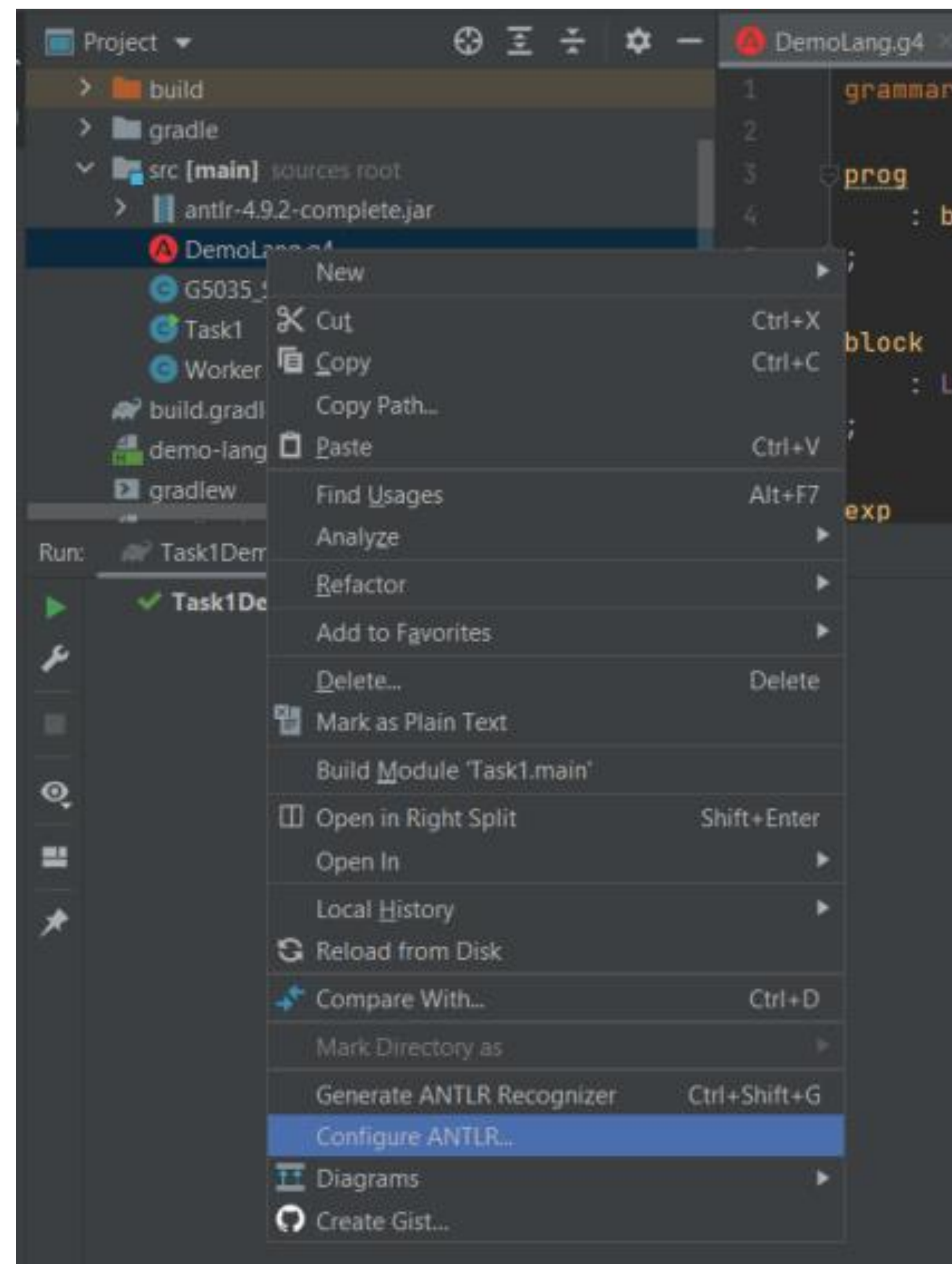
Using the ANTLR plugin to preview a Parse tree

Now that we have a **.g4** file, we can use the ANTLR IntelliJ Plugin to preview the parse tree that the ANLTR Lexer/Parser would generate for a given input. This is helpful for testing/debugging.

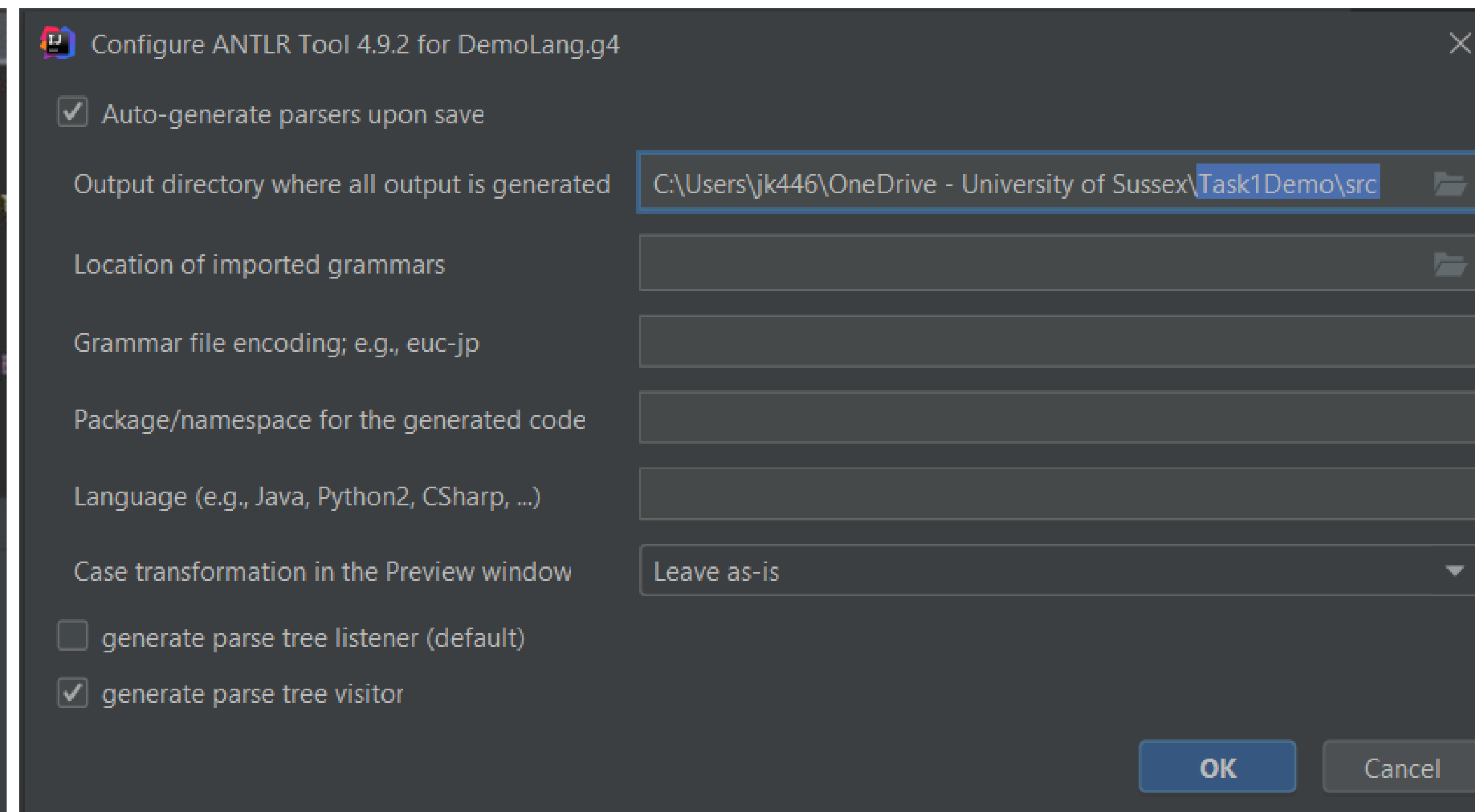


How do we get ANTLR to generate?

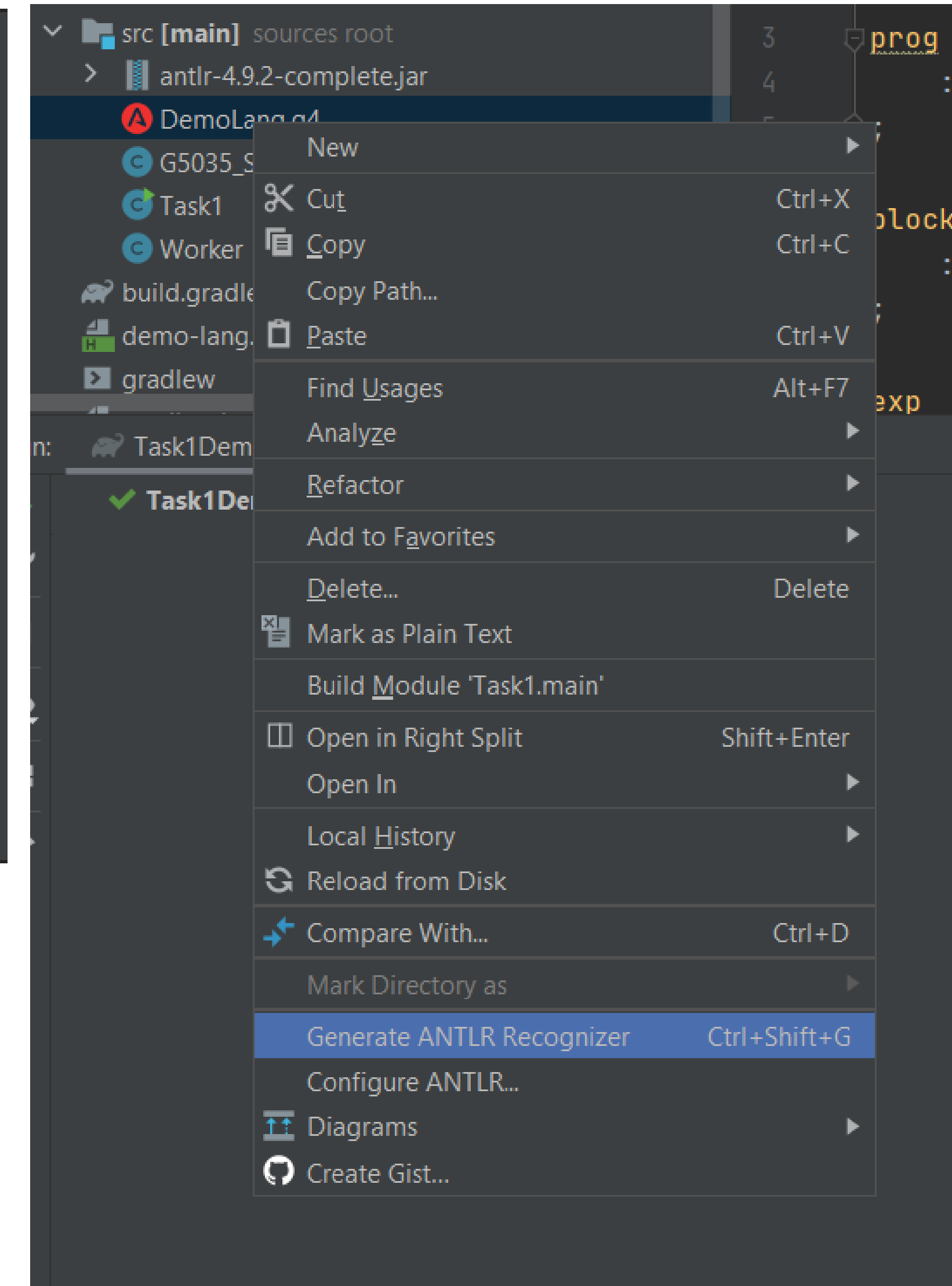
Right click the **.g4** file,
and click configure ANTLR



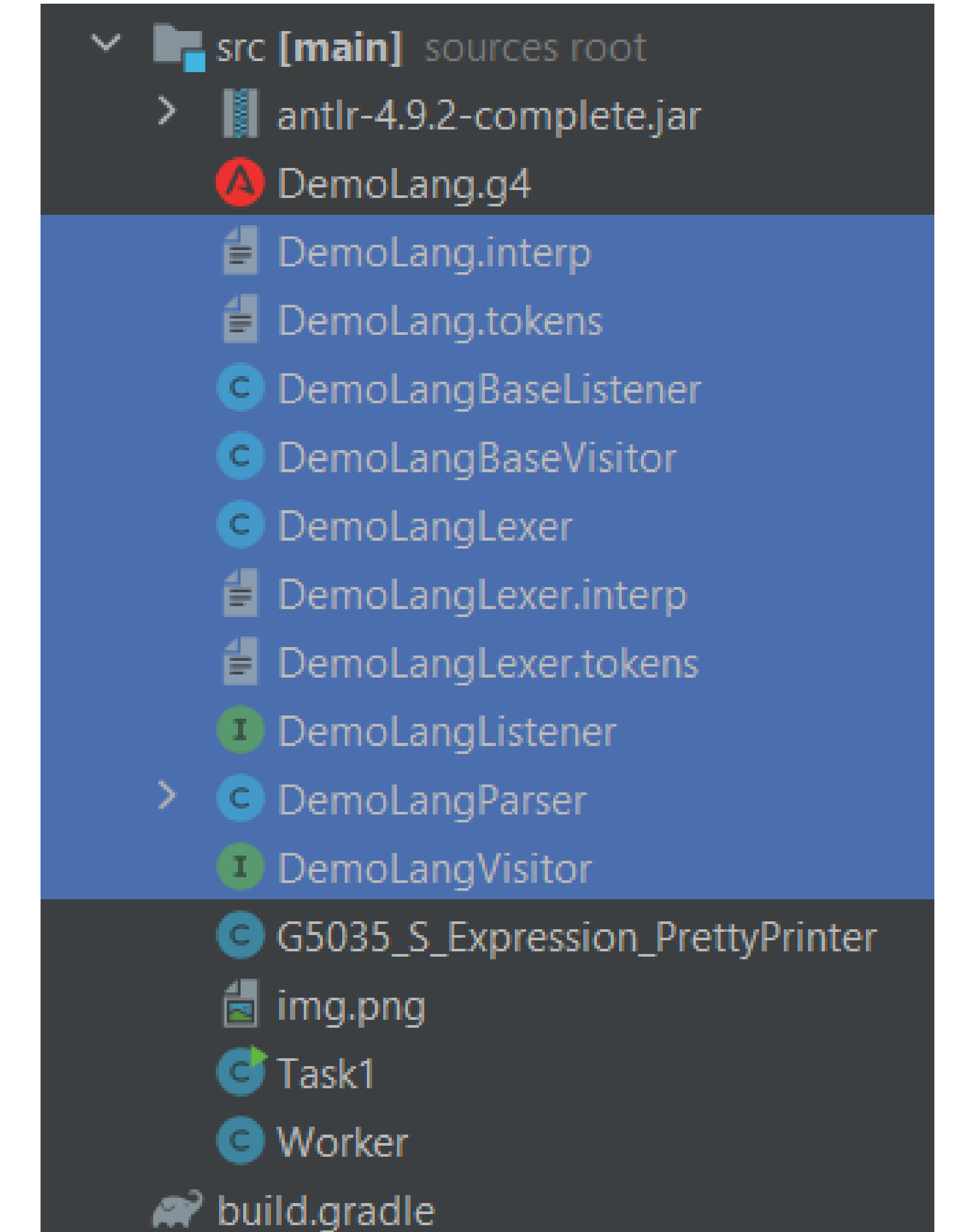
Tell ANTLR where to output
the generated files to (your src folder in your project)



Right click the **.g4** file,
and click generate ANTLR recognizer



ANTLR will generate the classes (you may
have to wait on the network drive!)



If you get stuck, look at lab classes: Week 1 Exercises 3-4, Week 2 Exercise 6, Week 3 Exercise 6, Week 4 Exercises 3-4, Week 5 Exercise 4.

How do we use the Lexer and Parser classes that ANTLR makes?

```
public static void main(String[] args) throws Exception {  
    // create a CharStream that reads from standard input  
    CharStream input = CharStreams.fromStream(System.in);  
  
    // create a lexer that feeds off of input CharStream  
    DemoLangLexer lexer = new DemoLangLexer(input);  
  
    // create a buffer of tokens pulled from the lexer  
    CommonTokenStream tokens = new CommonTokenStream(lexer);  
  
    // create a parser that feeds off the tokens buffer  
    DemoLangParser parser = new DemoLangParser(tokens);  
    ParseTree tree = parser.prog(); // begin parsing from non-terminal "prog"
```

In the example project, this is in file Task1.java

By default you can only visit the generalised top level of each rule definition

```
prog
: block EOF
;

block
: LBrace ene+=exp (Semicolon ene+=exp)* RBrace
;

exp
: Idfr Assign exp
| LParen exp binop exp RParen
| While exp Do block
| IntLit
| Idfr
```

```
/**
 * public class DemoLangBaseVisitor<T> extends AbstractParseTreeVisitor<T> implements DemoLangVisitor<T> {
 *     /** {@inheritDoc} ...*/
 *     @Override public T visitProg(DemoLangParser.ProgContext ctx) { return visitChildren(ctx); }
 *     /** {@inheritDoc} ...*/
 *     @Override public T visitBlock(DemoLangParser.BlockContext ctx) { return visitChildren(ctx); }
 *     /** {@inheritDoc} ...*/
 *     @Override public T visitExp(DemoLangParser.ExpContext ctx) { return visitChildren(ctx); }
 *     /**
```

But by adding #AltLabels to all the specific sub levels, you can also visit them individually instead

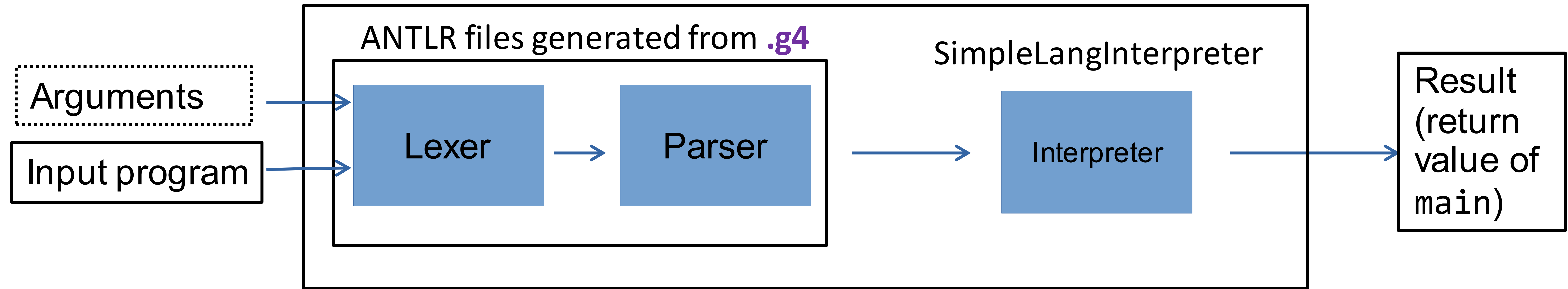
```
exp
: Idfr Assign exp
| LParen exp binop exp RParen
| While exp Do block
| IntLit
| Idfr

#AssignExpr
#BinOpExpr
#WhileExpr
#IntExpr
#IdExpr
```

```
Worker.java Task1.java build.gradle (Task1)
visitbinop
65 @ @ @Override public String visitIntExpr(DemoLangParser.Int
66 {
67     return "IntLit(" + ctx.IntLit().getText() + ")";
68 }
69
70 @ @ @Override public String visitIdExpr(DemoLangParser.IdE
71 {
72     return "Idfr(\"" + ctx.Idfr().getText() + "\")";
73 }
```

Overview of Task 1

Task1.java



1. Modify the **.g4** file and re-generate the ANTLR class files
2. Modify SimpleLangInterpreter.java
3. Test with the input programs provided (and more)
4. If anything fails – repeat steps 1 and/or 2
5. Once Items 1-4 are *perfect*, modify SimpleLangInterpreter.java to implement the type-checker too.